PLAYBOOK DE IMERSÃO

# Desenvolvimento Acelerado com IA

Da **engenharia de prompts** ao **Spec-Driven Development** em 30 páginas de teoria condensada, campos de treino e templates — para preencher durante a imersão e consultar pelo resto da carreira.

---

| | | |
|---|---|---|
| C | **Constitution** | estável · raro · o Mega Prompt do projeto |
| S | **Specify** | o quê e por quê · Given/When/Then |
| P | **Plan** | o como técnico · contexto para a IA |
| T | **Tasks** | atômicas · DoD verificável · chaining |
| I | **Implement** | execução com IA · review · self-reflection |

**Adriano Santos**

ESPECIALISTA EM DESENVOLVIMENTO DE SOFTWARE · MVP EMBARCADERO

30 PÁGINAS · 9 WORKSHEETS · 6 TEMPLATES

# Este não é um livro. É um caderno de campo.

*Livro se lê. Caderno se preenche. A diferença entre os dois é a diferença entre saber um método e usar um método — e é essa segunda que muda o seu jeito de programar.*

## § Três modos de uso

### ① Durante a imersão

Cada página de teoria é seguida de um **worksheet** com pautas para você preencher em tempo real. Traga caneta preta ou azul-escura — dourado no papel só nas capas. Os worksheets duram entre 10 e 25 minutos cada e valem mais que os slides.

### ② Nos 30 dias seguintes

Depois da imersão, faça o **Desafio dos 30 dias** (p. 28) aplicando o método a um projeto real seu. As páginas de teoria funcionam como referência rápida; os templates (p. 24–27) são para copiar e adaptar.

### ③ Como consulta permanente

Passado o desafio, o playbook vira **manual de bolso**. O Trilho do Método na margem direita ajuda a localizar em segundos onde você está no fluxo Constitution → Specify → Plan → Tasks → Implement.

### ④ Como material de time

Leve para as reuniões técnicas. Preencher um WS-04 (Constitution) ou WS-07 (Task com DoD) em conjunto, **projetado em tela**, alinha o time em minutos — sem precisar de nova ferramenta.

## § Convenções visuais

Teoria condensada · Worksheet · preencher · Kit · templates · Regra de ouro · Onde não confiar · Bloco de prompt

**◆ A REGRA DO CADERNO**

Playbook preenchido vale mais que playbook lido. Ao fim da imersão, **as suas respostas** — não as nossas páginas — são o material que você leva para o trabalho de segunda-feira. Se uma página ficar em branco, é porque aquele conceito ainda não é seu.

# Teoria à esquerda, sua caneta à direita

*Cada conceito ocupa uma página de teoria condensada seguida de um worksheet para preencher ao vivo.*


## ABERTURA p. 01-04

01 Capa
02 Como usar este playbook
03 Mapa do playbook
04 Manifesto — IA confiável > IA rápida

## BLOCO 1 · A LINGUAGEM p. 05-10

05 Os 4 Pilares do prompt
06 Treino dos 4 Pilares WS-01
07 4 técnicas essenciais
08 Reescreva com técnica WS-02
09 Mega Prompts & Self-Reflection
10 Monte seu Mega Prompt WS-03

## BLOCO 2 · O MÉTODO p. 11-20

11 Constitution — a carta magna
12 Suas 12 regras WS-04
13 Specify — descrever sem implementar
14 Sua user story G/W/T WS-05
15 Plan — engenharia sob controle
16 Suas decisões técnicas WS-06
17 Tasks — a ponte para a execução
18 Sua task com DoD WS-07
19 Implement — o ritual de execução
20 Checklist do ritual WS-08

## BLOCO 3 · O FLUXO p. 21-23

21 As 8 etapas do dev com IA
22 Onde não confiar na IA
23 Planeje sua próxima feature WS-09

## KIT DE TEMPLATES p. 24-27

24 K-01 · Esqueleto de Constitution
25 K-02 · Specify + Given/When/Then
26 K-03 · Task com DoD verificável
27 K-04 a K-06 · Prompts de bolso

## FECHAMENTO p. 28-30

28 Desafio dos 30 dias + rubrica
29 Suas anotações da imersão
30 Contracapa · próximos passos


◆ REGRA DE OURO

Não pule os worksheets. Preferimos que você faça 5 páginas de teoria e 5 de worksheet a 20 de teoria e nenhuma de prática — **a segunda combinação não funciona.**

# IA confiável > IA rápida.

*Sete princípios que atravessam este playbook. Se você discordar de algum, tudo bem — mas leia mesmo assim, porque o resto do material foi escrito acreditando neles.*

**01** **Velocidade sem confiabilidade é dívida técnica em modo turbo.**

Código gerado em 10 segundos e depurado em 3 horas custa mais que código escrito em 30 minutos e depurado em 5.

**02** **Contexto vence prompt esperto.**

Um prompt medíocre com contexto excelente ganha de um prompt brilhante com contexto ralo. Invista onde o retorno é maior.

**03** **A IA é literal. Explicitude não é rudeza — é engenharia.**

"Evite funções longas" é sugestão. "Funções não excedem 30 linhas" é regra. Adjetivo é rascunho; número é regra.

**04** **Erros da IA são sintomas.**

Quando o código gerado está errado, o defeito quase sempre está no artefato de origem — Constitution, Specify, Plan ou Task. Corrija a fonte, não o output.

**05** **Descrição a IA lê como sugestão; exemplo, como regra.**

Um exemplo bem escolhido vale mais que três parágrafos de explicação. Sempre que puder mostrar, mostre.

**06** **Sem números, o método vira religião.**

Meça: tempo economizado, bugs em produção, cobertura de testes, retrabalho. Sem baseline, não há método — há fé.

**07** **O método serve ao desenvolvedor, não o contrário.**

Se algum passo aqui não faz sentido no seu contexto, adapte. O que não pode faltar é a disciplina de *ter* um passo — não este passo específico.

## § 1 Os 4 Pilares de um bom prompt

A IA não lê sua mente — ela só tem acesso ao que você escreve. Quatro elementos transformam um prompt qualquer em um prompt eficiente. Não são truques: são pilares no sentido literal. **Se um deles falhar, o prompt inteiro desaba.**




O custo é desprezível: uns 90 segundos a mais formulando. O retorno é meia hora economizada em correção e retrabalho. Compare o mesmo pedido nas duas versões:

**● VAGO → ESTRUTURADO · O MESMO PEDIDO, DOIS RESULTADOS**

```
// vago — a IA escolhe uma interpretação (provavelmente a errada):
"Faz uma função que valida CPF"

// estruturado — os 4 pilares presentes:
Você é um desenvolvedor sênior. Preciso de uma função em [sua linguagem]
que valide CPF brasileiro.                          ← Contexto
Requisitos: aceitar com e sem máscara; rejeitar dígitos repetidos;
validar os 2 dígitos verificadores.                 ← Clareza
Retorne { valido, normalizado, motivo } + 5 casos de teste.   ← Formato
Sem dependências externas. Sem comentários óbvios.            ← Restrições
```

**◆ REGRA DE OURO DA CLAREZA**

Se **você** consegue interpretar seu próprio prompt de mais de uma forma, a IA também consegue — e pode escolher justamente a interpretação que não te serve.

**◇ ONDE NÃO CONFIAR**

Pilar 4 é o mais subutilizado. Pense nas respostas ruins que você já recebeu: o que tinha nelas que incomodou? **Transforme cada incômodo em restrição.** Essas micro-restrições, acumuladas, mudam dramaticamente o resultado.

**Agora é com você.** Vire a página com a caneta na mão: o worksheet ao lado é onde esses quatro pilares deixam de ser teoria e viram músculo. →

# WS-01
## Treino dos 4 Pilares

| 15 min | Caneta + LLM | Individual |
|---|---|---|
| DURAÇÃO | FERRAMENTAS | FORMATO |

**Missão:** pegue uma tarefa real da sua semana e escreva o prompt vago que você mandaria no piloto automático. Depois, reconstrua-o pilar por pilar. Ao final, envie as duas versões para a IA e compare.

## A O PROMPT NO PILOTO AUTOMÁTICO

// escreva do jeito que você mandaria hoje, sem pensar:

## B RECONSTRUA PILAR POR PILAR

### 1 CLAREZA *o que exatamente eu quero?*

### 2 CONTEXTO *quem sou, qual a situação?*

### 3 FORMATO *como quero receber?*

### 4 RESTRIÇÕES *o que eu NÃO quero?*

## C TESTE & COMPARE

// envie A e depois B para a mesma IA. O que mudou na resposta?

## ✓ AUTOCHECAGEM ANTES DE ENVIAR

- [ ] A tarefa tem uma única interpretação possível?
- [ ] Eu disse quem sou e qual o objetivo final?
- [ ] Especifiquei estrutura e tamanho da resposta?
- [ ] Eliminei pelo menos 2 armadilhas com restrições?

## § 2 As 4 técnicas essenciais

Sabendo escrever com os 4 Pilares, você acerta em quase tudo. Para os 20% de casos difíceis, existem **técnicas nomeadas** — atalhos batizados por quem já pisou nas mesmas pedras. Aqui vão as quatro que resolvem 90% do que sobra.

### T1 Role Prompting

Atribuir um papel muda o vocabulário, a profundidade e a postura da resposta. *"Você é um arquiteto de software com 15 anos de experiência"* muda mais o output do que 10 linhas de instrução.

QUANDO USAR → SEMPRE. É DE GRAÇA.

### T2 Few-Shot

Mostrar 2–3 exemplos do que você quer *antes* de pedir. A IA copia padrões melhor do que segue descrições. Vale para código, texto, JSON, commits — qualquer coisa que tenha forma.

QUANDO USAR → PADRÃO DE SAÍDA ESPECÍFICO OU TOM PARTICULAR

### T3 Chain of Thought

Pedir explicitamente *"pense passo a passo antes de responder"* força a IA a raciocinar em vez de chutar. Trocou uns segundos de espera por respostas dramaticamente melhores em problemas com múltiplos passos.

QUANDO USAR → LÓGICA, CÁLCULO, DEBUG, DECISÃO ARQUITETURAL

### T4 Output Structuring

Definir o esqueleto exato da resposta — JSON, tabela, seções nomeadas, cabeçalhos. Diminui interpretação, aumenta previsibilidade, torna a saída fácil de *parsear* por outro sistema.

QUANDO USAR → INTEGRAR COM CÓDIGO, GERAR ARTEFATOS, COMPARAR RESPOSTAS

◆ COMBINAÇÃO VALE MAIS QUE TÉCNICA ISOLADA

O jogo real é **empilhar**. Um Mega Prompt tipicamente usa Role + Few-Shot + Output Structuring de uma vez. Escolher só uma é sub-ótimo — as técnicas foram desenhadas para conviver, não para competir.

◆ REGRA DE OURO DO EXEMPLO

Descrição a IA lê como sugestão; exemplo, como regra. Se você tem 30 segundos para melhorar um prompt, gaste-os escolhendo **um** exemplo bem representativo em vez de reescrever a instrução.

◇ ONDE NÃO CONFIAR

Chain of Thought *parece* raciocínio, mas é geração de texto que *se parece* com raciocínio. Em problemas onde a resposta é matematicamente verificável, valide — não confie só na aparência de lógica.

# WS-02
## Reescreva com técnica

| 20 min | WS-01 pronto | Individual |
|---|---|---|
| DURAÇÃO | PRÉ-REQ | FORMATO |

**Missão:** pegue o prompt reconstruído no WS-01 (versão B) e produza **quatro variações**, cada uma aplicando uma técnica diferente. Envie as quatro para a IA e marque, ao final, qual serviu melhor ao seu caso.

### T1 ROLE PROMPTING · DEFINA UM PAPEL ESPECÍFICO

// "Você é um [função] com [X] anos de experiência em [domínio]..."

### T2 FEW-SHOT · DÊ 2 EXEMPLOS ANTES DE PEDIR

// "Entrada X → saída Y. Entrada X' → saída Y'. Agora, para minha entrada..."

### T3 CHAIN OF THOUGHT · PEÇA RACIOCÍNIO EXPLÍCITO

// "Antes de responder, pense passo a passo sobre..."

### T4 OUTPUT STRUCTURING · DEFINA O ESQUELETO EXATO DA RESPOSTA

// "Responda no formato: { campo1: ..., campo2: ..., campo3: ... }" ou tabela / seções

### ◆ COMPARAÇÃO

**T1 nota:** ___ / 10 **T2 nota:** ___ / 10 **T3 nota:** ___ / 10 **T4 nota:** ___ / 10

// A técnica vencedora foi ___ porque:

# § 3 Mega Prompts & Self-Reflection

Quando um prompt bem escrito não é o bastante, o próximo nível é o **Mega Prompt**: um documento estruturado que empilha técnicas em uma anatomia previsível. O que era "um pedido" vira "um contrato". Seis campos, sempre na mesma ordem:

**● A ANATOMIA DO MEGA PROMPT · 6 CAMPOS**

```
1. PAPEL        → Quem é a IA neste contexto (senioridade, especialidade, postura)
2. CONTEXTO     → O que ela precisa saber sobre o projeto, o time, o momento
3. CONTRATO     → O que exatamente deve ser entregue, com critérios de sucesso
4. REGRAS       → Restrições invioláveis (SEMPRE, NUNCA, PROIBIDO)
5. ENTREGA      → Formato e estrutura exata da resposta esperada
6. AUTO-CHECAGEM → Antes de responder, valide-se contra esta lista
```

O sexto campo é o que separa Mega Prompt de "prompt grande". Você pede à IA para **revisar a própria resposta antes de emiti-la** — é o *Self-Reflection*. Não é mágico: força um segundo passe interno que pega omissões, contradições e violações de regra que passariam batido. Custa 5% mais de latência; corta 40% dos retornos ruins.

### O loop que fecha o método

Mega Prompts e SDD não são coisas separadas. Um Mega Prompt bem feito *é* a materialização, em texto, dos artefatos do SDD:




**◆ REGRA DE OURO DO MEGA PROMPT**

Se você abre o mesmo tipo de conversa com a IA **três vezes por semana**, é hora de virar Mega Prompt. O que era improviso repetido vira template repetível — e o que era template vira, com o tempo, comando ou agente.

**◇ ONDE NÃO CONFIAR**

Self-Reflection não corrige violação de regra que você *não escreveu*. Ele só verifica contra o que está no prompt. Uma regra crítica omitida é uma regra crítica ausente — não peça à IA para "usar o bom senso" onde você mesmo não foi explícito.

# WS-03
# Monte seu Mega Prompt

| 20 min | Um caso recorrente | Individual |
|---|---|---|
| DURAÇÃO | BASE | FORMATO |

**Missão:** escolha *uma* tarefa que você repete com a IA (revisão de PR, geração de teste, refactor, documentação, migração...) e escreva o Mega Prompt correspondente. No fim, salve-o como snippet, macro, slash-command — qualquer coisa que dê para reusar sem redigitar.

O caso escolhido é:

## 1 PAPEL *quem é a IA aqui?*

## 2 CONTEXTO *o que ela precisa saber?*

## 3 CONTRATO *o que deve ser entregue?*

## 4 REGRAS *SEMPRE / NUNCA / PROIBIDO*

## 5 ENTREGA *estrutura exata da resposta*

## 6 AUTO-CHECAGEM *valide-se contra estes itens*

◆ PRÓXIMO PASSO — NÃO PULE

Onde este Mega Prompt vai morar? Em **Notes**? **Snippet** da IDE? **Custom instructions** do ChatGPT/Claude? **CLAUDE.md** do projeto? Marque abaixo — Mega Prompt sem endereço vira Mega Prompt perdido.

// Endereço definitivo deste Mega Prompt:

## § 4 Constitution — a carta magna

A Constitution é o primeiro pilar do SDD e o mais estável de todos. São as **regras invioláveis** do projeto: aquilo que nunca muda entre uma feature e outra, entre um dev e outro, entre uma execução da IA e a próxima. É o Mega Prompt do projeto inteiro — escrito uma vez, lido antes de cada task.




Os seis blocos abaixo mapeiam um a um para as seções de um system prompt profissional. Cada regra recebe um **prefixo rastreável** — o mesmo identificador que aparecerá depois nas Tasks, nos commits e nos comentários do código gerado:

| PREFIXO | BLOCO | O QUE DETERMINA |
|---|---|---|
| AR- | **Arquitetura** | Camadas, fronteiras, quem fala com quem, padrões estruturais |
| QC- | **Qualidade** | Limites numéricos, proibições explícitas, padrões de nomenclatura |
| ST- | **Stack** | Bibliotecas aprovadas e vetadas, versões, políticas de dependência |
| CO- | **Compatibilidade** | Plataformas, runtimes, encodings, versões mínimas suportadas |
| GO- | **Governança** | Branches, commits, quem aprova o quê, quando escalonar |
| SE- | **Segurança** | Dados sensíveis, criptografia, logging, segredos |

● CONSTITUTION.MD · TRECHO EXECUTÁVEL

```
# CONSTITUTION — [Seu Projeto]   v1.0 · aprovada pelo tech lead

## QC — Qualidade de Código
QC-01: Funções não devem exceder 30 linhas.
QC-02: PROIBIDO: capturar exceções genéricas silenciando o erro.
QC-03: Toda entrada externa passa por validação antes da lógica.
```

◆ A REGRA QUE MUDA TUDO

A Constitution não educa — **ela determina**. Não é "a gente prefere que", nem "sugerimos que". É o texto que aparece antes de cada execução da IA e que define, em termos verificáveis, o que passa e o que não passa. Se uma regra depende de bom senso, ela não é uma regra: é um convite ao caos.

# WS-04

## Suas 12 regras — Constitution mínima

| 25 min | Seu projeto | Dupla |
|---|---|---|
| DURAÇÃO | BASE | FORMATO |

**Missão:** duas regras por bloco, cada uma com critério *mensurável* — número, lista ou proibição. Lembre: **a Constitution não educa, ela determina.** Adjetivo é rascunho; número é regra.

| BLOCO | REGRA 1 — CRITÉRIO MENSURÁVEL | REGRA 2 |
|---|---|---|
| AR- Arquitetura | | |
| QC- Qualidade | | |
| ST- Stack | | |
| CO- Compatib. | | |
| GO- Governança | | |
| SE- Segurança | | |

◆ PROVA DE FOGO

Troque o playbook com a dupla ao lado. Para cada regra do colega, pergunte: **"uma IA conseguiria verificar isso sozinha?"** Se a resposta for não, a regra volta para reescrita — com número, lista ou PROIBIDO explícito. Marque abaixo quantas regras suas passaram na primeira: ___/12.

# § 5 Specify — descrever sem implementar

Se a Constitution responde *"quais são as regras do jogo?"*, o Specify responde *"o que exatamente estamos construindo agora?"*. É onde produto e engenharia se encontram: o **quê** e o **por quê** — nunca o como. O como fica para o Plan.

### A anatomia mínima de uma Specify

Não precisa ser um documento longo. Precisa ser **preciso**. Uma Specify utilizável tem quatro seções:

| SEÇÃO | PERGUNTA QUE RESPONDE | CUIDADO PRINCIPAL |
|---|---|---|
| **Contexto** | Por que estamos fazendo isso agora? | Motivo real, não desejo genérico |
| **User story** | Como usuário X, quero Y, para Z | Papel + ação + benefício |
| **Cenários G/W/T** | Quando isso funcionou de verdade? | Verificáveis pela IA |
| **Fora de escopo** | O que NÃO faremos agora? | Corta a criatividade excessiva |

### Given / When / Then · o formato executável

É a peça mais importante da Specify — e a que a maioria dos times pula. Um cenário G/W/T bem escrito descreve um comportamento observável em três linhas. E ele vira, quase palavra por palavra, o esqueleto do teste automatizado:

● GIVEN / WHEN / THEN · O CRITÉRIO QUE VIRA TESTE

```
Cenário: Cliente resgata cupom de primeira compra
  GIVEN um cliente sem histórico de pedidos
  AND   um cupom válido "BEMVINDO10" ativo até 31/12
  WHEN  ele aplica o cupom no checkout
  THEN  10% de desconto é aplicado ao total
  AND   o cupom fica marcado como usado para este cliente
```

Perceba: nenhuma menção a banco de dados, framework, arquitetura ou nome de classe. A Specify descreve o **comportamento**; o Plan decide como isso se implementa.

◆ REGRA DE OURO DA SPECIFY

Se você consegue escrever a Specify e passá-la para **outro desenvolvedor** que produza uma implementação razoável, ela está pronta. Se ele precisa perguntar "o que você quis dizer com...", volte à Specify — não à implementação.

◇ ONDE NÃO CONFIAR

User story sozinha não é Specify. "Como usuário, quero fazer login" gera 47 interpretações diferentes de IA. Sem cenários G/W/T e sem "fora de escopo", você terceiriza a decisão de produto para o modelo. **Você não quer isso.**

# WS-05
## Sua Specify — user story + G/W/T

| 20 min | Uma feature real | Individual |
|---|---|---|
| DURAÇÃO | BASE | FORMATO |

**Missão:** escolha uma feature pequena e real (algo que caberia em 1–3 dias de trabalho) e escreva a Specify completa. Ao final, entregue a versão preenchida para um colega e peça: *"consegue implementar sem me perguntar nada?"*. Ajuste até a resposta ser sim.

### A CONTEXTO · POR QUE AGORA?

// qual dor real motiva esta feature? qual métrica move?

### B USER STORY · PAPEL · AÇÃO · BENEFÍCIO

// Como [papel], quero [ação], para [benefício verificável].

### C CENÁRIO 1 · GIVEN / WHEN / THEN — O CAMINHO FELIZ

### C' CENÁRIO 2 · GIVEN / WHEN / THEN — UM CASO DE BORDA

### D FORA DE ESCOPO · O QUE NÃO FAREMOS AGORA

// listar aqui protege o time do "aproveitando que estamos mexendo…"

# § 6 Plan — engenharia sob controle

Se Constitution é *o que sempre vale* e Specify é *o que faremos agora*, o Plan é **como faremos agora**. Não é um documento de UML nem um caderno de anotações: é o conjunto mínimo de decisões técnicas que a IA precisa saber para não inventar arquitetura por conta própria.

## O que entra no Plan

Cinco categorias de decisão, sempre explicitadas — e sempre com **alternativa considerada**. Uma decisão sem alternativa é fé; com alternativa, é engenharia:

| CATEGORIA | O QUE DECIDIR | FORMATO MÍNIMO |
|---|---|---|
| **Arquitetura local** | Camadas, módulos e fronteiras *desta* feature | Diagrama ou lista de módulos |
| **Fluxo de dados** | Entrada → transformações → saída, incluindo persistência | Fluxograma ou pseudocódigo |
| **Contratos de interface** | Assinaturas públicas: parâmetros, tipos, erros possíveis | Assinaturas em texto |
| **Dependências novas** | Bibliotecas a adicionar, com justificativa vs. alternativas | Lista com "por quê" |
| **Riscos & incógnitas** | O que pode dar errado, o que ainda não sabemos | Lista curta e honesta |

## Plan é contexto para a IA

Sem Plan, a IA extrapola. Ela vai inventar uma arquitetura, escolher uma biblioteca famosa e adivinhar a interface. Provavelmente vai errar em uma dessas três — e o "erro" custa caro, porque parece funcionar. Com Plan, você **restringe** o espaço de escolhas antes de pedir código. É a diferença entre "faça login" e "implemente esta interface, com estas dependências, seguindo este fluxo".

● PLAN.MD · TRECHO REAL

```
## Feature: Resgate de cupom no checkout

### Arquitetura local
- CouponService (Domain)  → validação e aplicação de regra
- CouponRepo (Infra)      → persistência e marcação de uso
- CheckoutController       → orquestra chamadas

### Dependência nova
- Nenhuma. Reaproveitar DateProvider existente.
  ↳ alternativa: adicionar biblioteca de datas · descartada por AR-02
```

◆ REGRA DE OURO DO PLAN

Se o Plan tem **zero** "alternativas descartadas" documentadas, ele está mentindo. Ninguém constrói uma feature sem considerar dois caminhos. Registrar o descartado é o que evita revisitar a mesma decisão em três meses.

# WS-06

## Suas decisões técnicas — Plan enxuto

| 20 min | Specify do WS-05 | Individual |
|---|---|---|
| DURAÇÃO | PRÉ-REQ | FORMATO |

**Missão:** usando a Specify que você escreveu no WS-05, produza o Plan técnico. Toda decisão precisa listar **uma alternativa descartada** — sem isso, não é decisão, é chute.

### A ARQUITETURA LOCAL · MÓDULOS E RESPONSABILIDADES DESTA FEATURE

### B FLUXO DE DADOS · ENTRADA → TRANSFORMAÇÕES → SAÍDA

### C CONTRATOS · ASSINATURAS PÚBLICAS (PARÂMETROS, RETORNOS, ERROS)

### D DECISÃO + ALTERNATIVA DESCARTADA

Decisão tomada:

Alternativa considerada e descartada porque:

### E RISCOS & INCÓGNITAS · O QUE PODE DAR ERRADO?

// honestidade > otimismo. IA não resolve o que você não nomeia.

# § 7 Tasks — a ponte para a execução

Constitution, Specify e Plan produzem juntos uma coisa: **tasks**. Cada task é uma unidade atômica de trabalho — pequena o suficiente para caber em uma execução da IA, grande o suficiente para produzir valor sozinha. Se está com dúvida sobre o tamanho, o teste é este: *uma task deveria terminar em uma única conversa com a IA, sem trocar de contexto.*

### Os 6 campos de uma task utilizável

A task é o Mega Prompt em formato de ticket. Os mesmos seis campos que você viu no WS-03, agora com identidade rastreável:

**● T-042 · APLICAR CUPOM NO CHECKOUT**

```
ID          T-042
Referência  Specify #C-007 · Plan §2 · Constitution [QC-01, ST-03]
Contexto    Feature de resgate de cupom, cenário do caminho feliz.
Ação        Implementar CouponService.apply(cart, code) conforme
            contratos definidos no Plan §2.C.
Restrições  Sem novas dependências. Retornar Result<Discount, Error>.
DoD         ✓ Cenário G/W/T #1 verde
            ✓ Cupom marcado como usado após aplicação
            ✓ Cobertura ≥ 90% nas linhas alteradas
            ✓ Zero warnings do linter
```

### DoD — Definition of Done é o "sim, terminei"

DoD é o que separa "achou que terminou" de "terminou". Cada item da DoD tem que ser **verificável em segundos**, seja por olho, por teste ou por ferramenta. Se um item da DoD depende de julgamento subjetivo, ele não é DoD — é desejo.

**◆ REGRA DE OURO DA TASK**

Task boa cabe em uma tela. Se a sua ocupa três, ela é **três tasks**. Divida. O ganho de rastreabilidade compensa o "trabalho extra" de decompor — que, na prática, é o próprio trabalho de pensar.

**◆ CHAINING · A SEQUÊNCIA COMO PODER**

Uma task raramente vive só. O poder está em **encadeá-las**: T-042 gera o service, T-043 escreve os testes, T-044 integra ao controller, T-045 documenta no README. Cada uma é uma execução limpa da IA. O erro de uma não contamina a próxima — porque cada uma parte da mesma Constitution, do mesmo Plan, dos mesmos contratos.

**◇ ONDE NÃO CONFIAR**

Task sem DoD é convite ao "ta bom pra mim". A IA vai concordar que terminou, você vai concordar que terminou, e três dias depois o QA vai descobrir que ninguém tinha testado o caso vazio. **DoD escrita antes da execução** é o que evita isso.

# WS-07
## Sua primeira task com DoD verificável

| 15 min | Plan do WS-06 | Individual |
|---|---|---|
| DURAÇÃO | PRÉ-REQ | FORMATO |

**Missão:** tire a *primeira* task do seu Plan. Pequena, atômica, terminável em uma conversa com a IA. E escreva a DoD antes de executar — não depois.

ID DA TASK

REFERÊNCIAS

### A CONTEXTO · ONDE ESSA TASK SE ENCAIXA NO FLUXO?

### B AÇÃO · O QUE EXATAMENTE SERÁ FEITO?

// verbo forte + objeto específico. "Implementar X". "Adicionar Y". "Refatorar Z".

### C RESTRIÇÕES · O QUE NÃO FAZER NESTA TASK?

### D DOD · DEFINA ANTES DE EXECUTAR (CADA ITEM VERIFICÁVEL EM SEGUNDOS)

- [ ] _____
- [ ] _____
- [ ] _____
- [ ] _____

**◆ TESTE DE ATOMICIDADE**

Leia sua task em voz alta. Se você precisou dizer *"aí depois eu..."*, ela não é atômica. Corte no "aí depois" e vire duas tasks. Rastreabilidade > comodidade.

## § 8 Implement — o ritual de execução

Aqui a IA entra em campo. Todos os pilares anteriores existiram para tornar *este* momento previsível: a IA lê Constitution + Specify + Plan + Task, produz código, você valida contra a DoD, ajusta o que falhou. Se algo deu errado, quase nunca é culpa da IA — é sinal de que um artefato anterior está impreciso.

### O ritual em 5 passos, sempre nesta ordem

| PASSO | AÇÃO | VERIFICAÇÃO |
|---|---|---|
| 01 | **Contextualizar** | Constitution, Specify, Plan e Task na janela da IA |
| 02 | **Executar** | Prompt final = "Execute T-XXX seguindo os artefatos acima" |
| 03 | **Validar contra DoD** | Cada item da DoD checado — não "parece pronto" |
| 04 | **Rodar testes** | Testes automatizados verdes antes de qualquer commit |
| 05 | **Registrar aprendizado** | Erro da IA → qual artefato precisa refinar? |

### Self-Reflection na execução

Antes de aceitar o código gerado, faça o mesmo passe do Mega Prompt: peça à IA para **revisar a própria saída contra a DoD**. Não confie — verifique também. Mas peça primeiro, porque em 40% dos casos ela mesma encontra o problema.

● PROMPT DE REVIEW · COLE APÓS RECEBER O CÓDIGO

```
Antes de eu revisar, faça você um self-review honesto:

1. Para cada item da DoD, marque ✓ ou ✗ e explique a evidência.
2. Aponte 2 pontos que você mesma mudaria se refizesse.
3. Liste toda linha que assume algo não declarado no Plan.
4. Se você acha que terminou, escreva "PRONTO". Senão, "AJUSTAR".
```

◆ CORRIJA A FONTE, NÃO O OUTPUT

A IA gerou algo errado três vezes seguidas? Não é a IA. É a Constitution vaga, ou a Specify ambígua, ou o Plan incompleto. Volte, refine **a fonte**, regenere. O próximo código já nasce correto — e todo próximo caso similar também.

◇ ONDE NÃO CONFIAR

Código que compila e testes que passam **não são** DoD. São dois itens dela. Se sua DoD tem quatro critérios e você validou dois, você entregou 50% — não 100%. Marcar como pronto sem checar todos é dívida técnica com juros.

# WS-08

## Checklist do ritual — sua ficha de execução

| 10 min | Repetível | Por task |
|---|---|---|
| DURAÇÃO | USO | APLICAÇÃO |

**Missão:** execute a task do WS-07 usando esta ficha como roteiro. Depois, tire uma foto da página preenchida — este é o gabarito visual que você vai reproduzir mentalmente todas as próximas vezes.

### 01 CONTEXTUALIZAR · ARTEFATOS NA JANELA DA IA?

- ☐ Constitution v___ carregada
- ☐ Specify #____ colada/anexada
- ☐ Plan §____ colado/anexado
- ☐ Task T-____ é a única no foco

### 02 EXECUTAR · REGISTRE O PROMPT FINAL QUE VOCÊ ENVIOU

### 03 VALIDAR · MARQUE CADA ITEM DA DOD

- ☐ DoD item 1: _____ evidência: _____
- ☐ DoD item 2: _____ evidência: _____
- ☐ DoD item 3: _____ evidência: _____
- ☐ DoD item 4: _____ evidência: _____

### 04 TESTES · TESTES AUTOMATIZADOS RODARAM E PASSARAM?

☐ Sim, todos verdes ☐ Não — qual falhou:

### 05 APRENDIZADO · ALGUM ARTEFATO PRECISA REFINAR?

```
// "quando a IA errou X, era porque a Constitution/Specify/Plan/Task não deixava claro Y"
```

C S P T I

## § 9 As 8 etapas do dev com IA

O ciclo completo, do requisito ao deploy, passa por oito momentos onde a IA *pode* ajudar — e onde o método muda a cada um. Este é o mapa que você vai desenhar sobre seus fluxos reais.

| # | ETAPA | O QUE A IA FAZ BEM AQUI | ARTEFATO DOMINANTE |
|---|---|---|---|
| 01 | **Descoberta** | Explorar o problema, gerar hipóteses, mapear stakeholders | `Specify (rascunho)` |
| 02 | **Especificação** | Converter conversas em user stories, gerar cenários G/W/T | `Specify` |
| 03 | **Arquitetura** | Comparar padrões, propor divisões de módulos, listar trade-offs | `Plan` |
| 04 | **Decomposição** | Quebrar Plan em tasks atômicas com DoD verificável | `Tasks` |
| 05 | **Codificação** | Gerar implementação, sugerir refactor, aplicar padrões | `Task + código` |
| 06 | **Testes** | Escrever unit/integration, cobrir bordas, gerar dados de teste | `Task + testes` |
| 07 | **Review** | Detectar smells, revisar segurança, verificar DoD | `Prompt de review` |
| 08 | **Documentação** | Gerar README, changelog, ADR a partir dos artefatos existentes | `Todos os anteriores` |

◆ O PADRÃO INVISÍVEL

Nas 8 etapas, o mesmo movimento se repete: **alimente a IA com os artefatos anteriores**. Codificação lê Task + Plan + Constitution. Review lê Task + código + Constitution. Documentação lê tudo. Quem pula essa alimentação está usando IA como oráculo, não como colega — e o resultado é oráculo.

◆ REGRA DE OURO DO FLUXO

Nem toda etapa precisa de IA. Descoberta com stakeholders humanos é humana; documentação de ADR a partir dos artefatos é IA. O critério é: se você **já** escreveu o insumo em algum artefato, a IA transforma bem. Se não escreveu, use humano.

## § 10 Onde não confiar na IA

Confiança calibrada > confiança cega > desconfiança total. O objetivo desta página é a primeira: saber, com precisão, os lugares onde a IA *parece* certa mas costuma errar. São dez armadilhas conhecidas. Marque as que já te pegaram — e volte aqui antes de cada task que caia numa delas.

### A1 Alucinação de API

Métodos, argumentos e endpoints que *parecem* existir mas não existem. Sempre verifique documentação oficial em vez de acreditar.

### A2 Versão desatualizada

Código correto para a versão de *ontem*, quebrado hoje. Explicite a versão da lib na Constitution.

### A3 Segurança superficial

Sanitização "razoável" que não cobre injeção real. Toda camada de segurança precisa de revisão humana explícita.

### A4 Testes que não testam

Asserts que passam com qualquer input. Sempre inspecione os assertions gerados — não confie no verde.

### A5 Estimativa otimista

"Deve levar uns 15 minutos". A IA subestima consistentemente. Multiplique por 3 e por experiência.

### A6 Refactor invasivo

Pediu para trocar o nome de um método, ela reescreveu o arquivo. Sempre restrinja escopo do refactor no prompt.

### A7 Contexto do negócio

Regras de domínio específicas ("no Brasil, boleto vence em dia útil"). Fora da Constitution, ela chuta.

### A8 Otimização prematura

Cache, paralelismo, índice — inseridos sem medir. Peça explicitamente para **não** otimizar até haver métrica.

### A9 Concordância excessiva

Se você propõe algo errado, ela costuma concordar. Force contraposição: "quais são os 3 pontos fracos desta proposta?".

### A10 Fronteiras de sistemas

Integrações com legado, protocolos raros, hardware específico. Zona onde a IA improvisa mais e acerta menos.

◆ A META-REGRA

Se você **não conseguiria** revisar aquilo por conta própria em um review humano, terceirizar para a IA é gerenciar risco cego. A IA acelera onde você *já sabe*. Ela é multiplicador — se o multiplicando é zero, o resultado é zero.

# WS-09

## Planeje sua próxima feature — integrador

| 30 min | Feature real | Individual |
|---|---|---|
| DURAÇÃO | BASE | FORMATO |

**Missão:** este é o worksheet integrador — traz os cinco pilares em uma única página. Escolha uma feature que você vai começar *na semana que vem* e planeje-a inteira aqui. Ao fim da imersão, esta página vira o seu plano de ação real.

A feature é:

### C CONSTITUTION · QUAIS REGRAS EXISTENTES SE APLICAM? PRECISO CRIAR/REFINAR ALGUMA?

### S SPECIFY · USER STORY + 1 CENÁRIO G/W/T PRINCIPAL

### P PLAN · A DECISÃO TÉCNICA-CHAVE (COM ALTERNATIVA DESCARTADA)

### T TASKS · LISTE AS 3–5 PRIMEIRAS TASKS, CADA UMA EM UMA LINHA

### I IMPLEMENT · COMO VOU MEDIR SE O MÉTODO VALEU A PENA?

// tempo? bugs? cobertura? satisfaçao? escolha 1 metrica e um numero-alvo.

# K-01 Esqueleto de Constitution

*Copie, adapte e versione em* `/docs/spec/CONSTITUTION.md`. *Agnóstico de linguagem — campos entre colchetes são seus.*

**● TEMPLATE · CONSTITUTION EM 6 BLOCOS**

```
# CONSTITUTION — [nome do projeto]
Versão: [1.0.0]   Aprovação: [tech lead]   Revisão: via PR, sempre

## AR — Arquitetura
AR-01: Camadas obrigatórias: [Presentation, Application, Domain, Infra].
AR-02: [Quem NUNCA fala com quem — ex.: UI nunca acessa banco direto].

## QC — Qualidade de Código
QC-01: Funções não excedem [30] linhas.
QC-02: PROIBIDO: [lista explícita — ex.: catch genérico, tipos dinâmicos].
QC-03: Validação obrigatória em toda entrada externa, com [ferramenta].

## ST — Stack Tecnológica
ST-01: Aprovadas: [libs com versão]. Vetadas: [libs sem manutenção 12m+].

## CO — Compatibilidade
CO-01: Plataformas: [SO/browser/runtime mínimos]. Encoding: [UTF-8].

## GO — Governança
GO-01: [Fluxo de branch]. PR obrigatório com [N] aprovador(es).
GO-02: Mudança nesta Constitution exige [aprovação formal de quem].

## SE — Segurança
SE-01: NUNCA registrar dado sensível em log: [lista do seu domínio].
SE-02: Segredos apenas em [vault/variável de ambiente], jamais no código.
```

**◆ RITUAL DE USO — 5 MINUTOS QUE ECONOMIZAM HORAS**

Antes de disparar a IA para qualquer task: ① Constitution atualizada e em contexto? ② Specify cobre o caso? ③ Plan tem as decisões técnicas? ④ Task tem DoD verificável? Se algum item falhar — pause, refine, só então execute.

**◇ LEMBRETE DO MÉTODO**

Erros da IA são quase sempre sintomas de falhas nos artefatos. Em vez de corrigir o código gerado, **corrija a fonte** — o próximo código já nasce correto.

# K-02 Specify + Given/When/Then

*Uma folha por feature. Se ficou grande demais para uma folha, a feature está grande demais para uma iteração.*

**● TEMPLATE · SPECIFY.MD**

```
# SPECIFY — [C-###] [nome curto da feature]
Autor: [nome]   Data: [aaaa-mm-dd]   Status: rascunho | revisão | aprovado

## 1. Contexto
Por que agora? Qual dor real? Qual métrica move?
[3-5 linhas · direto ao ponto · sem venda]

## 2. User story
Como [papel], quero [ação observável], para [benefício verificável].

## 3. Cenários (Given / When / Then)

### Cenário 3.1 — caminho feliz
  GIVEN [estado inicial verificável]
  WHEN  [ação observável]
  THEN  [resultado observável]
  AND   [efeito colateral verificável]

### Cenário 3.2 — caso de borda
  GIVEN [estado atípico]
  WHEN  [ação]
  THEN  [comportamento esperado — erro? fallback? bloqueio?]

## 4. Fora de escopo
- [item 1 que ninguém deve implementar aproveitando esta feature]
- [item 2 que fica para outra iteração]

## 5. Métricas de sucesso
- [métrica 1] passa de [X] para [Y] em [prazo]
- [métrica 2] · nova linha de base
```

**◆ O TESTE DO COLEGA**

Entregue este Specify a um dev que não estava na conversa. Se ele consegue implementar sem te perguntar nada, **pronto**. Se precisou de qualquer clarificação, aquela clarificação é o próximo item a entrar na Specify — não a resposta oral do momento.

# K-03 Task com DoD verificável

*Cada task cabe em uma tela. Cada DoD é verificável em segundos. Cada referência é rastreável ao artefato de origem.*

**● TEMPLATE · TASK T-###**

```
ID:          T-[###]
Título:      [verbo forte + objeto específico]
Referências: Specify #[C-###] · Plan §[X.Y] · Constitution [prefixos]
Estimativa: [1h | 2h | 4h | ½ dia]  ← se > ½ dia, divida

## Contexto
[2-3 linhas de onde essa task se encaixa no fluxo maior]

## Ação
[o que exatamente será feito, em uma linha imperativa]

## Entradas & Saídas
- Entrada:  [dados/interfaces que a task consome]
- Saída:    [artefato produzido — código, testes, doc]

## Restrições
- SEM [dependência/refactor não-solicitado/mudança fora do escopo]
- MANTER [contratos existentes / assinaturas / naming]

## Definition of Done
[ ] [critério 1 verificável em segundos, com evidência clara]
[ ] [critério 2 · teste automatizado passa]
[ ] [critério 3 · cobertura ≥ N% nas linhas alteradas]
[ ] [critério 4 · zero warnings/lint erros novos]

## Notas para a IA (opcional)
[padrão a seguir, exemplo próximo no repo, armadilha conhecida]
```

◆ **REGRA DE OURO DO TEMPLATE**

Uma task com todos os campos preenchidos **vira Mega Prompt em segundos**: cole os artefatos referenciados + a task, adicione uma linha "execute T-XXX seguindo os artefatos acima" e envie. É o formato que fecha o loop teoria → método → execução.

# Prompts de bolso

*Três prompts pequenos, testados, que você pode salvar como snippet hoje e usar para o resto da carreira. Ajuste o texto entre colchetes ao seu contexto.*

## K-04 Review de Pull Request

**● COLE APÓS O DIFF**

```
Você é um revisor sênior. Revise o diff acima contra [Constitution].
Estruture como: ✓ Aprovado · △ Sugestões · ✗ Bloqueios.
Cada bloqueio cita a regra violada (AR-XX, QC-XX...). Sem elogio genérico.
```

## K-05 Refactor com escopo restrito

**● COLE COM O ARQUIVO ALVO**

```
Refatore apenas [função/método/classe X] em [arquivo Y].
Objetivo: [reduzir aninhamento | extrair método | renomear | ...].
NÃO alterar assinaturas públicas. NÃO tocar outros arquivos.
Devolva: (1) diff proposto, (2) 2 linhas explicando por quê.
```

## K-06 Debug estruturado

**● COLE COM O ERRO + CONTEXTO**

```
Estou vendo [erro/comportamento] em [contexto: arquivo/rota/fluxo].
Comportamento esperado: [o que deveria acontecer].
Antes de propor solução, faça: Chain of Thought em 5 passos.
1. Reproduzir mentalmente o fluxo. 2. Listar 3 hipóteses. 3. Para cada uma,
qual evidência confirmaria/refutaria. 4. Ordenar pela mais provável.
5. Sugerir o próximo experimento pequeno — não a solução final.
```

**◆ SEU KIT VAI CRESCER**

Estes três são um começo. À medida que você repetir prompts no dia-a-dia, promova cada padrão para snippet (K-07, K-08, K-09...). O objetivo é chegar em ~15 prompts salvos que cobrem 80% do seu trabalho recorrente. **É esse arsenal — não a ferramenta de IA — que faz sua velocidade.**

# 30 dias. Um projeto real.

*Terminar a imersão é o começo. O que vem agora — 30 dias aplicando o método a um projeto seu — é o que faz o playbook virar hábito.*

| SEMANA | OBJETIVO | ENTREGÁVEL | ☐ |
|---|---|---|---|
| Semana 1 | **Fundação** | Constitution v1.0 do projeto, com 12 regras versionadas em Git | |
| Semana 2 | **Primeira feature** | Specify + Plan + 3 tasks com DoD, executadas com IA seguindo o ritual | |
| Semana 3 | **Refinamento** | Constitution v1.1 com regras descobertas na execução; 1 template próprio (K-07) | |
| Semana 4 | **Retrospectiva & escala** | Métricas coletadas; método apresentado ao time; 1 colega adotando | |

### Rubrica de autoavaliação · onde você chegou?

Ao fim dos 30 dias, marque o nível que descreve honestamente seu estado. Sem trapacear — a única pessoa que perde com isso é você.

| NÍVEL | COMO SABER QUE VOCÊ CHEGOU AQUI |
|---|---|
| ◇ PRATA | Constitution existe e é usada; pelo menos 1 feature entregou com o método completo; comecei a medir tempo e retrabalho. |
| ◆ OURO | Método aplicado em *toda* nova feature; 3+ templates próprios criados; erros da IA diminuíram porque estou corrigindo os artefatos, não o output. |
| ✦ OURO C/ DIAMANTE | Time inteiro adotou; Constitution é revisada em ritual mensal; você mede baseline vs. atual e o método vira parte da cultura de engenharia — não do seu esforço pessoal. |

**◆ COMPROMISSO DE FECHAMENTO**

Assine embaixo. Escrever o nome à mão neste playbook é um contrato com você mesmo — e é surpreendentemente mais eficaz do que qualquer app de metas.

Nome: _____ Data: ___ / ___ / _____

# O que você vai lembrar em 6 meses

*Página em branco de propósito. Aqui vão as coisas que você não quer esquecer — insight, frase que grudou, decisão de mudar, coisa que discordou. Escreva em cima da hora, não no fim do dia.*

### ◆ INSIGHTS · O QUE ME IMPACTOU?

### → AÇÕES · O QUE VOU FAZER SEGUNDA-FEIRA

### ◇ DISCORDÂNCIAS · ONDE NÃO CONCORDEI

### ? DÚVIDAS · O QUE FICOU PENDENTE

◆ RITUAL DE REVISITA

Reveja esta página **em 30 dias**. Depois **em 90 dias**. O que era ação e virou hábito? O que era dúvida e virou clareza? O que era insight e ficou só no papel? Reajuste — e escreva a data ao lado de cada item que revisitou.

FIM DO CADERNO · COMEÇO DO HÁBITO

# Você não terminou este playbook. Você começou a usar o método.

Da próxima vez que a IA gerar algo que não te serve, volte às páginas 05, 11, 13, 15, 17 e 19. Uma delas — quase sempre — tem a resposta do que ficou impreciso. Este caderno é seu diagnóstico portátil. Preencheu tudo? Passe adiante para um colega que precise.

---


SOBRE O AUTOR

## Adriano Santos

Especialista em desenvolvimento de software, com carreira dedicada a Delphi e evolução para IA generativa em engenharia. MVP Embarcadero. Autor de *SPEC-Driven Development — um guia para o presente* e *A Linguagem Secreta da IA — Engenharia de Prompts para quem constrói software de verdade*, os dois livros que deram origem a este playbook.


PRÓXIMOS TREINAMENTOS

- ◆ Desenvolvimento Acelerado com IA · turmas mensais
- ◆ Constitution para times · workshop de 4h
- ◆ SDD aplicado a Delphi · avançado
- ◆ Mentoria individual · 90 dias


adriano@adrianosantos.com.br
adrianosantostreinamentos.com.br



Playbook de imersão · 1ª edição · 2026 · PT-BR · impressão A4